

ALGORITHMES ET SYSTÈMES DISTRIBUÉS

Cours de préparation — S7 M1 Informatique (ILIA)

Coordonner des processus qui ne partagent ni mémoire ni horloge commune : horloges logiques, exclusion mutuelle, élection, consensus et tolérance aux pannes — avec exercices corrigés.

Sommaire

1. Introduction aux systèmes distribués
2. Modèles de communication
3. Horloges logiques et causalité
4. Exclusion mutuelle distribuée
5. Élection de leader
6. Consensus et tolérance aux pannes
7. Exercices récapitulatifs

Chapitre 1 — Introduction aux systèmes distribués

Un **système distribué** est un ensemble de processus, souvent répartis sur des machines différentes, qui coopèrent pour réaliser une tâche commune en échangeant des messages sur un réseau. Contrairement à un programme séquentiel classique, on perd deux garanties essentielles : une **horloge globale** partagée, et une vision instantanée de l'état de tous les processus.

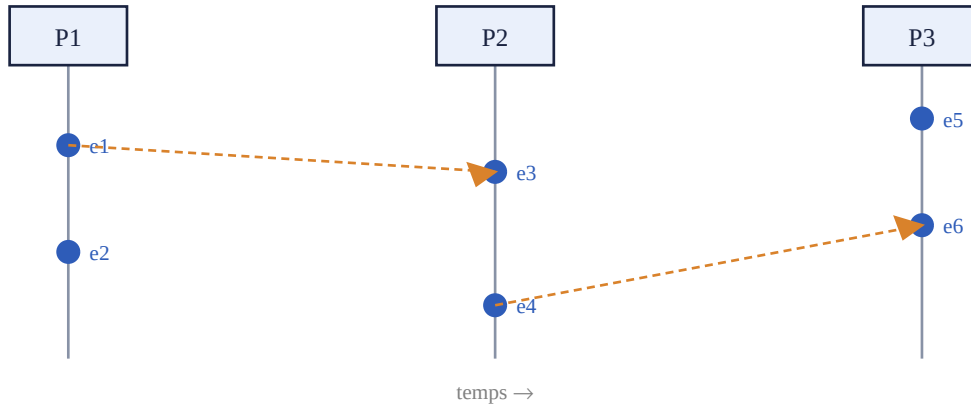


Figure 1.1 — Trois processus, chacun avec sa propre horloge locale, communiquant par messages

1.1 Les difficultés propres au distribué

- **Absence d'horloge globale** : on ne peut pas dater deux événements sur deux machines différentes de façon fiable avec l'heure physique seule.
- **Pannes partielles** : un processus ou un lien réseau peut tomber en panne sans que les autres le sachent immédiatement — on ne peut pas distinguer facilement « lent » de « en panne ».
- **Asynchronisme** : les messages peuvent arriver avec un délai variable, voire être perdus ou dupliqués selon le modèle considéré.

À retenir : concevoir un algorithme distribué, c'est raisonner avec seulement ce que chaque processus voit localement (ses propres événements et les messages reçus), jamais avec une vue globale instantanée du système.

Ce chapitre pose les bases ; les chapitres suivants donnent des outils concrets (horloges logiques, algorithmes d'exclusion mutuelle, d'élection, de consensus) pour reconstruire de l'ordre et de la coordination malgré ces contraintes.

Chapitre 2 — Modèles de communication

2.1 Passage de messages vs mémoire partagée

Deux grands modèles de communication existent entre processus distribués :

Modèle	Principe	Exemple
Mémoire partagée	Les processus lisent/écrivent dans une zone mémoire commune (nécessite du matériel ou un système partagé)	Threads sur une même machine multicœur
Passage de messages	Les processus s'échangent des messages explicites via le réseau, aucune mémoire commune	Microservices, systèmes répartis sur plusieurs machines

Dans un vrai système distribué (plusieurs machines), c'est le modèle par **passage de messages** qui domine — c'est celui qu'on étudie dans ce cours.

2.2 Modèles de synchronisme

- **Synchrone** : on connaît une borne maximale sur le temps de transmission d'un message et sur la vitesse d'exécution des processus. Plus simple à raisonner, mais irréaliste sur Internet.
- **Asynchrone** : aucune borne de temps garantie. C'est le modèle le plus réaliste, mais aussi le plus difficile : certains problèmes (comme le consensus) y deviennent impossibles à résoudre de façon garantie en présence de pannes (résultat FLP).

FLP (Fischer, Lynch, Paterson, 1985) : dans un système asynchrone, il est impossible de garantir le consensus de façon déterministe si ne serait-ce qu'un seul processus peut tomber en panne. En pratique, on contourne cette limite avec des mécanismes comme les timeouts ou le hasard (algorithmes randomisés).

Exercice 1 — Modèles de communication

- a) Pourquoi le modèle « mémoire partagée » est-il rarement utilisé tel quel entre deux machines physiquement distantes ?
- b) Donnez un exemple concret où le modèle synchrone est une hypothèse raisonnable, et un exemple où elle ne l'est clairement pas.

Correction

a) Il n'existe pas de mémoire physique partagée entre deux machines distinctes : toute « mémoire partagée » distante devrait en réalité être simulée par des messages réseau, ce qui réintroduit les délais et pannes qu'on cherchait justement à éviter — et ajoute une couche de complexité et de latence.

b) Raisonnable : un bus de terrain industriel avec des délais de transmission bornés et garantis. Pas raisonnable : une communication entre deux serveurs sur Internet, où la latence peut varier fortement selon la charge du réseau.

Chapitre 3 — Horloges logiques et causalité

Puisqu'on ne dispose pas d'horloge physique commune fiable, on utilise des **horloges logiques** pour ordonner les événements de façon cohérente avec la causalité : si un événement e_1 peut avoir influencé un événement e_2 , on veut que l'horloge de e_1 soit strictement inférieure à celle de e_2 .

3.1 Horloge de Lamport

Chaque processus maintient un compteur entier H , incrémenté à chaque événement local :

1. Avant un événement local : $H = H + 1$
2. Avant d'envoyer un message : $H = H + 1$, on estampille le message avec H
3. À la réception d'un message estampillé H_m : $H = \max(H, H_m) + 1$

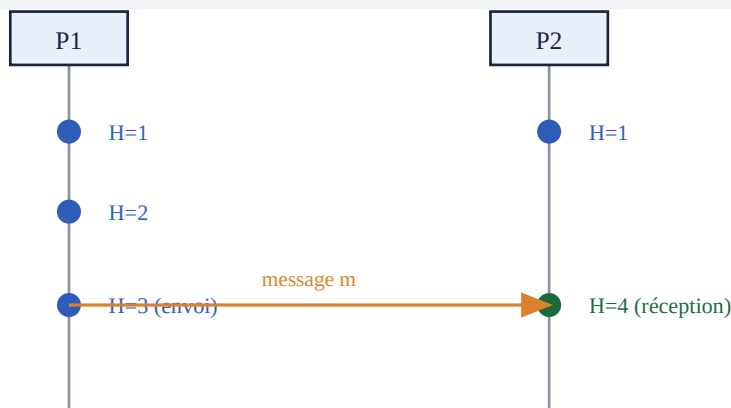


Figure 3.1 — Horloges de Lamport : P1 envoie un message estampillé $H=3$, P2 le reçoit et passe à $H=\max(1,3)+1=4$

Limite de l'horloge de Lamport : si $H(e_1) < H(e_2)$, cela ne veut **pas** forcément dire que e_1 a causalement influencé e_2 — la relation ne fonctionne que dans un sens. Pour capturer complètement la causalité, il faut des **vecteurs d'horloges**.

3.2 Vecteurs d'horloges

Chaque processus P_i maintient un vecteur V de taille n (nombre de processus), où $V[i]$ compte ses propres événements et $V[j]$ ($j \neq i$) sa meilleure estimation de l'horloge de P_j . Avec les vecteurs d'horloges, on peut décider précisément si deux événements sont causalement liés ou concurrents (indépendants).

Exercice 2 — Horloges de Lamport

Deux processus P_1 et P_2 démarrent avec $H=0$. P_1 exécute 2 événements locaux (H passe à 1 puis 2), puis envoie un message m à P_2 (H passe à 3, m est estampillé 3). Au moment de la réception, P_2 avait déjà exécuté 1 événement local ($H=1$). Quelle est la valeur de H chez P_2 après réception de m ?

Correction

Règle de réception : $H = \max(H_{\text{local}}, H_{\text{message}}) + 1 = \max(1, 3) + 1 = 4$. L'horloge de P_2 passe donc à 4 après réception du message.

Chapitre 4 — Exclusion mutuelle distribuée

L'**exclusion mutuelle** garantit qu'un seul processus à la fois accède à une ressource critique. En centralisé, un simple verrou suffit ; en distribué, il faut un protocole d'échange de messages puisqu'aucun processus n'a une vue globale instantanée.

4.1 Algorithme centralisé (par coordinateur)

Un processus **coordinateur** unique gère une file d'attente de requêtes. Pour entrer en section critique, un processus lui envoie une requête et attend une autorisation ; en sortant, il envoie un message de libération.

- **Avantage** : simple, garantit l'exclusion mutuelle et l'équité (ordre FIFO possible).
- **Inconvénient** : point unique de défaillance (si le coordinateur tombe en panne, plus personne ne peut entrer en section critique) et goulot d'étranglement.

4.2 Algorithme de Ricart-Agrawala (distribué)

Chaque processus souhaitant entrer en section critique diffuse une requête horodatée (horloge de Lamport) à tous les autres processus, et attend l'accord de tous avant d'entrer. Un processus retarde sa réponse s'il est lui-même en section critique, ou s'il a une requête plus ancienne en attente.

Comparaison d'horodatage : en cas de requêtes concurrentes, on compare les couples (horloge de Lamport, identifiant du processus) pour départager de façon déterministe — cela évite les situations d'interblocage entre deux requêtes de même horloge.

Exercice 3 — Exclusion mutuelle

Deux processus P3 et P5 envoient une requête d'entrée en section critique estampillée du même temps logique $H=7$. Selon la règle $(H, \text{identifiant})$, lequel entre en premier en section critique ? Justifiez.

Correction

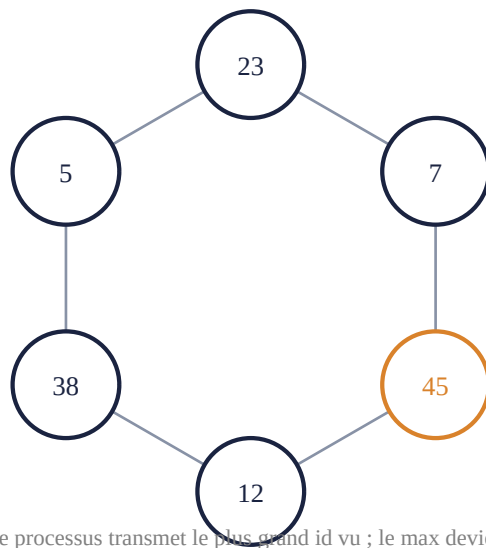
En cas d'égalité des horloges de Lamport, on départage par l'identifiant du processus (convention généralement : le plus petit identifiant est prioritaire). P3 entre donc en premier, car $3 < 5$. Cette règle garantit un ordre total et déterministe, connu de tous les processus.

Chapitre 5 — Élection de leader

De nombreux algorithmes distribués ont besoin d'un processus « coordinateur » ou « leader » pour simplifier une tâche (voir chapitre précédent). L'**élection** est le problème consistant à désigner ce leader de façon cohérente : tous les processus doivent s'accorder sur le même élu.

5.1 Algorithme en anneau (Chang-Roberts)

Les processus sont organisés logiquement en anneau. Chaque processus envoie son identifiant à son voisin ; à réception d'un identifiant, un processus le transmet seulement s'il est plus grand que le sien, sinon il transmet le sien à la place. Le plus grand identifiant fait ainsi le tour complet de l'anneau et devient le leader reconnu par tous.



Anneau : chaque processus transmet le plus grand id vu ; le max devient leader (ici 45)

Figure 5.1 — Élection en anneau : l'identifiant maximal (45) circule et devient leader

5.2 Algorithme du « bully » (brutal)

Chaque processus connaît l'identifiant de tous les autres. Quand un processus détecte que le leader est en panne, il envoie une requête d'élection à tous les processus d'identifiant supérieur. S'il ne reçoit aucune réponse, il se déclare leader ; sinon, un processus supérieur reprend la main. C'est simple mais génère beaucoup de messages.

Exercice 4 — Élection en anneau

Cinq processus sont disposés en anneau avec les identifiants suivants, dans l'ordre : 12 → 34 → 9 → 41 → 20 → (retour à 12). Le processus 9 initie une élection. Décrivez la circulation des messages et indiquez le leader final.

Correction

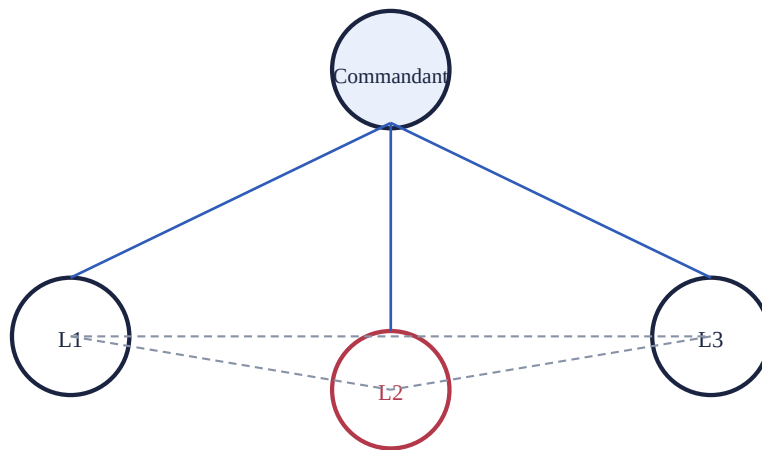
9 envoie son id à 41. $41 > 9$ donc 41 transmet son propre id (41) à 20. $20 < 41$, donc 20 transmet 41 (pas son propre id) à 12. $12 < 41$, transmet 41 à 34. $34 < 41$, transmet 41 à 9. 9 reconnaît son propre point de départ n'a pas encore vu 41... En pratique, l'id 41 fait un tour complet et revient à 41 lui-même, qui se déclare leader et diffuse l'information à tout l'anneau. Le leader final est donc le processus 41 (le plus grand identifiant).

Chapitre 6 — Consensus et tolérance aux pannes

Le **consensus** demande à un ensemble de processus de se mettre d'accord sur une seule valeur commune, même si certains processus tombent en panne ou envoient des informations incohérentes.

6.1 Le problème des généraux byzantins

Un commandant doit transmettre un ordre (« attaquer » ou « reculer ») à plusieurs lieutenants. Certains participants (commandant ou lieutenants) peuvent être **traîtres** : ils envoient délibérément des informations différentes à différents destinataires pour semer la confusion. On veut malgré tout que tous les lieutenants loyaux exécutent le même ordre.



Le lieutenant traître peut envoyer des ordres contradictoires

Figure 6.1 — Un lieutenant traître peut transmettre des ordres contradictoires

Résultat clé : avec n participants dont f traîtres, un accord byzantin fiable n'est possible que si $n \geq 3f + 1$. Autrement dit, il faut plus des deux tiers de participants loyaux.

6.2 Tolérance aux pannes « simples » (crash)

Le cas des pannes par arrêt (crash, sans comportement malveillant) est plus simple que le cas byzantin : un processus en panne cesse simplement de répondre, il n'envoie jamais d'informations incohérentes. Des algorithmes comme Paxos ou Raft (hors programme détaillé ici, mais bons mots-clés à connaître) résolvent le consensus dans ce contexte, à condition d'avoir une majorité de processus fonctionnels.

Exercice 5 — Consensus byzantin

Un système compte 4 participants. Combien de traîtres au maximum le système peut-il tolérer tout en garantissant un consensus byzantin fiable ? Justifiez avec la formule du cours.

Correction

On veut $n \geq 3f + 1$, soit $f \leq (n-1)/3 = (4-1)/3 = 1$. Le système peut donc tolérer au maximum **1 traître** parmi les 4 participants pour garantir un consensus fiable. Avec 2 traîtres ou plus, l'accord byzantin ne peut plus être garanti.

Chapitre 7 — Exercices récapitulatifs

Exercice 6 — Synthèse — de la causalité au consensus

Trois processus P1, P2, P3 collaborent. P1 envoie un message à P2, qui après réception envoie à son tour un message à P3.

- a) Pourquoi ne peut-on pas simplement utiliser l'heure système de chaque machine pour ordonner ces trois événements de façon fiable ?
- b) Quel outil du cours permettrait de garantir que l'horloge logique de l'événement chez P3 soit supérieure à celle chez P1 ?
- c) Si l'un des trois processus peut mentir sur les messages qu'il retransmet, quel type de problème doit-on résoudre pour que les deux autres s'accordent malgré tout ?

Correction

- a) Les horloges physiques de machines différentes ne sont jamais parfaitement synchronisées (dérive, latence réseau variable) : on ne peut pas garantir que l'ordre des timestamps physiques respecte l'ordre causal réel des événements.
- b) L'horloge de Lamport (ou un vecteur d'horloges pour plus de précision) : la règle de réception $H = \max(H_local, H_message) + 1$ garantit que chaque message reçu fait avancer l'horloge du destinataire au-delà de celle de l'émetteur.
- c) Il s'agit d'un problème de **consensus byzantin**, puisqu'un participant peut se comporter de façon malveillante et incohérente plutôt que simplement tomber en panne.

Exercice 7 — Question ouverte

Expliquez pourquoi l'algorithme centralisé d'exclusion mutuelle (chapitre 4) est plus simple que Ricart-Agrawala, mais moins adapté à un système à grande échelle avec de fortes exigences de disponibilité.

Correction

L'algorithme centralisé concentre toute la logique dans un seul coordinateur, ce qui simplifie grandement le raisonnement et l'implémentation (un seul point de décision, pas besoin de comparer des horloges logiques). Mais ce coordinateur devient un point unique de défaillance (single point of failure) : s'il tombe en panne, plus aucun processus ne peut obtenir l'accès à la ressource critique, et il peut aussi devenir un goulot d'étranglement si le nombre de requêtes augmente. Ricart-Agrawala, bien que plus complexe, répartit la décision entre tous les processus et n'a pas ce point de défaillance unique.